

Git Challenges

1. Resolve Merge Conflicts

- Create a merge conflict intentionally (two users editing the same line).
- Resolve the conflict and push the changes.

Objective:

Create a merge conflict intentionally and resolve it successfully.

Purpose:

To understand how Git handles conflicting changes made to the same line of a file.

Implementation:

1. Created a Git repository and committed the initial file.
2. Created a feature branch.
3. Modified the same line in both main and the feature branch.
4. Attempted to merge the feature branch into main.
5. Identified the merge conflict using git status.
6. Manually resolved the conflicting lines.
7. Used git add and committed the resolved changes.
8. Pushed the updated main branch to GitHub

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (hotfix)
$ git status
On branch hotfix
Your branch is up to date with 'origin/hotfix'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   alphabet

no changes added to commit (use "git add" and/or "git commit -a")

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (hotfix)
$ git add .
warning: in the working copy of 'alphabet', LF will be replaced by CRLF t

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (hotfix)
$ git commit -m "modified alphabet file"
[hotfix 693a67f] modified alphabet file
1 file changed, 1 insertion(+)
```

```
MINGW64/c/Users/91738/Desktop/techie-02
$ git push origin hotfix
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 275 bytes | 275.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:vamshikumar-s/techie-02.git
 c9595ef..693a67f hotfix -> hotfix

$ git checkout release
Switched to branch 'release'
Your branch is up to date with 'origin/release'.

$ git pull
Already up to date.

$ ls
README.md a b release

$ nano alphabet
hello horizon

$ git status
On branch release
Your branch is up to date with 'origin/release'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    alphabet

nothing added to commit but untracked files present (use "git add" to track)

$ git add .
warning: in the working copy of 'alphabet', LF will be replaced by CRLF the next time Git touches it

$ git push origin release
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 285 bytes | 285.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:vamshikumar-s/techie-02.git
 63f3bb..add6ca3 release -> release
```

```
WARNING: in the working copy of 'alphabet', LF will be replaced by CRLF the next time Git touches it

$ git commit -m "adding alphabet file"
[release add6ca3] adding alphabet file
1 file changed, 1 insertion(+),
create mode 100644 alphabet

$ git merge hotfix
Auto-merging alphabet
CONFLICT (add/add): Merge conflict in alphabet
Automatic merge failed; fix conflicts and then commit the result.

$ git status
On branch release
Your branch is up to date with 'origin/release'.

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

changes to be committed:
  modified:   a
  renamed:   b -> file2.txt
  new file:   test
  new file:   webhook-test.txt

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both added:   alphabet

$ nano alphabet
```

```
MINGW64:/c/Users/91738/Desktop/techie-02
GNU nano 6.3
<<<<<<< HEAD
hello horizon
=====
hello techie
>>>>>>> hotfix
```

```
MINGW64/c/Users/91738/Desktop/techie-02
On branch release
Your branch is up to date with 'origin/release'.

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Changes to be committed:
  modified:   1
  renamed:    b -> file2.txt
  new file:   test
  new file:   webhook-test.txt

Unmerged paths:
  (use "git add <file>..." to mark resolution)
  both added:   alphabet

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release|MERGING)
$ git add .
$ git commit -m "resolved the merge conflict of alphabet file"
[release 9331056] resolved the merge conflict of alphabet file

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git push origin release
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 339 bytes | 339.00 KiB/s, done.
Total 3 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:vamshikumar-a/techie-02.git
   ad8c6a3..9331056  release -> release

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git log
commit 93310563a1c4ed85492be03836dbb61ced4073af (HEAD -> release, origin/release)
Merge: ad8c6a3 693a67f
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 13:35:22 2026 +0530

    resolved the merge conflict of alphabet file

commit ad8c6a38a0a71334228cebaa62ae42bb16bdcf21
Author: vamshikumar-a <vamshikumara1004@gmail.com>
```

Commits

🔗 release ▾

🔍 Commits on Aug 21, 2026

resolved the merge conflict of alphabet file

👤 vamshikumar-a committed 1 minute ago

adding alphabet file

👤 vamshikumar-a committed 10 minutes ago

modified alphabet file

👤 vamshikumar-a committed 16 minutes ago

Conclusion:

Successfully created and resolved a Git merge conflict.

2. Recover Deleted Branch

- Delete a local branch and then recover it using the reflog.

Objective:

Recover a deleted local Git branch using git reflog.

Purpose:

To understand how to recover a branch that was accidentally deleted.

Implementation:

1. Created a new branch and added a commit.
2. Switched back to the main branch.
3. Deleted the local branch using git branch -D.

4. Used git reflog to find the deleted branch's latest commit.
5. Created a new branch using the recovered commit ID.
6. Verified the recovered branch and its commits.

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git branch feature

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git branch --list
  feature
  hotfix
  main
* release

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git push origin release
Everything up-to-date

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git checkout feature
Switched to branch 'feature'

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ ls
README.md  a  alphabet  file2.txt  release  test  webhook-test.txt

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ nano test

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git status
On branch feature
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test

no changes added to commit (use "git add" and/or "git commit -a")
```

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git branch -D feature
Deleted branch feature (was f61b107).

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git branch --list
  hotfix
  main
* release

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git reflog
9331056 (HEAD -> release, origin/release) HEAD@{0}: checkout: moving from feature to release
f61b107 HEAD@{1}: commit: modified test file
9331056 (HEAD -> release, origin/release) HEAD@{2}: checkout: moving from release to feature
```

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git checkout -b feature f61b107
Switched to a new branch 'feature'

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git branch --list
* feature
  hotfix
  main
  release
```

Conclusion:

Successfully recovered a deleted branch using Git reflog.

3. Undo Wrong Push

- Push a wrong commit to GitHub, then undo it without losing history.

Objective:

Undo an incorrect commit that was already pushed to GitHub without removing its history.

Purpose:

To safely reverse an incorrect change in a shared repository.

Implementation:

1. Created an incorrect change and committed it.
2. Pushed the commit to GitHub.
3. Identified the incorrect commit using git log.
4. Used git revert <commit-id> to create a new commit that reverses the change.
5. Pushed the revert commit to GitHub.
6. Verified that the original commit remained in the Git history.

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ ls
README.md  a  alphabet  file2.txt  release  test  webhook-test.txt

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ nano demo.txt

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ ls
README.md  a  alphabet  demo.txt  file2.txt  release  test  webhook-test.txt

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git status
On branch feature
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        demo.txt

nothing added to commit but untracked files present (use "git add" to track)
```

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git add .
warning: in the working copy of 'demo.txt', LF will be replaced by CRLF the next time Git touches it

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git commit -m "added demo file"
[feature 5343321] added demo file
1 file changed, 1 insertion(+)
create mode 100644 demo.txt
```

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git push origin feature
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 277 bytes | 277.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:vamshikumar-a/techie-02.git
   f61b107..5343321  feature -> feature

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git log
commit 5343321c929d4b8dfb73f584bddc0bd529fb4a6b (HEAD -> feature, origin/feature)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date:   Fri Aug 21 14:50:54 2026 +0530

    added demo file

commit f61b107bae48f6730d27566db2d210af6d4d34d0
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date:   Fri Aug 21 14:39:51 2026 +0530

    modified test file
```

```
MINGW64/c:/Users/91738/Desktop/techie-02
Date: Thu Aug 20 17:46:10 2026 +0530

Merge branch 'hotfix' of github.com:vamshikumar-a/techie-02 into hotfix

commit da5e7b4d6638125bd4aba321ae88d87a6efaac70
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Thu Aug 20 17:44:23 2026 +0530

    testing github webhooks

commit e56f493d181a680f6d5a36eeeb45c245c0580cb
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Thu Aug 20 15:42:18 2026 +0530

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ ls
README.md a alphabet demo.txt file2.txt release test webhook-test.txt
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git revert 534332
[feature 92201ef] Revert "added demo file"
1 file changed, 1 deletion(-)
delete mode 100644 demo.txt
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ ls
README.md a alphabet file2.txt release test webhook-test.txt
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git log
commit 92201ef028fe5e9d068ec2637b91ceeb189a0cf8 (HEAD -> feature)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 14:52:56 2026 +0530

    Revert "added demo file"

    This reverts commit 5343321c929d4b8dfb73f584bdc0bd529fb4a6b.

commit 5343321c929d4b8dfb73f584bdc0bd529fb4a6b (origin/feature)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 14:50:54 2026 +0530

    added demo file

commit f61b107bae48f6730d27566db2d210af6d4d34d0
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 14:39:51 2026 +0530
```

Conclusion:

Successfully reverted an incorrect pushed commit without losing Git history.

4. Amend a Commit

- Make a commit, then add a missing file to it using git commit --amend.

Objective:

Add a missing file to the latest Git commit using git commit --amend.

Purpose:

To correct the latest commit without creating an additional commit.

Implementation:

1. Created and committed the first file.
2. Created another file that was accidentally missed.
3. Added the missing file using git add.
4. Used git commit --amend --no-edit.
5. Verified that both files were included in the same commit.

```
MINGW64/c/Users/91738/Desktop/techie-02
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ ls
README.md  a  alphabet  file2.txt  release  test  webhook-test.txt
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ nano amend.txt
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git status
On branch feature
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    amend.txt

nothing added to commit but untracked files present (use "git add" to track)
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git add .
warning: in the working copy of 'amend.txt', LF will be replaced by CRLF the next time Git touches it
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git commit -m "adding amend file"
[feature fb18651] adding amend file
1 file changed, 1 insertion(+)
create mode 100644 amend.txt
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ nano amend2.txt
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git status
On branch feature
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    amend2.txt

nothing added to commit but untracked files present (use "git add" to track)
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git add .
warning: in the working copy of 'amend2.txt', LF will be replaced by CRLF the next time Git touches it
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git commit --amend
[feature fb18651] adding amend files
Date: Fri Aug 21 15:09:24 2026 +0530
2 files changed, 2 insertions(+)
```

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git commit --amend
[feature fb18651] adding amend files
Date: Fri Aug 21 15:09:24 2026 +0530
2 files changed, 2 insertions(+)
create mode 100644 amend.txt
create mode 100644 amend2.txt

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git log
commit fb18651a2bcd2aa6f8f947fadb7e0e942d9f6556 (HEAD -> feature)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 15:09:24 2026 +0530

    adding amend files
```

```
adding amend files

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Fri Aug 21 15:09:24 2026 +0530
#
# On branch feature
# Changes to be committed:
#   new file:   amend.txt
#   new file:   amend2.txt
#
~
~
~
```

```

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git show
commit fb18651a2bcd2aa6f8f947fadb7e0e942d9f6556 (HEAD -> feature)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 15:09:24 2026 +0530

    adding amend files

diff --git a/amend.txt b/amend.txt
new file mode 100644
index 0000000..08f7258
--- /dev/null
+++ b/amend.txt
@@ -0,0 +1 @@
+this is first amend file
diff --git a/amend2.txt b/amend2.txt
new file mode 100644
index 0000000..4f1c2ba
--- /dev/null
+++ b/amend2.txt
@@ -0,0 +1 @@
+this is second amend file
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ |

```

Commit fb18651

vamshikumar-a committed 2 minutes ago

[Browse files](#)

adding amend files

[feature](#)

1 parent [92201ef](#) commit fb18651

2 files changed

+2

Filter files...

amend.txt
amend2.txt

Search within code

```

amend.txt
+++ @@ -0,0 +1 @@
1 + this is first amend file

amend2.txt
+++ @@ -0,0 +1 @@
1 + this is second amend file

```

Conclusion:

Successfully amended the latest commit with the missing file.

5. Cherry-pick a Commit

- Take a specific commit from one branch and apply it to another branch.

Objective:

Apply a specific commit from one branch to another branch.

Purpose:

To selectively apply a particular change without merging the entire branch.

Implementation:

1. Created a feature branch.
2. Made and committed a change on the feature branch.
3. Obtained the commit ID using git log.
4. Switched to the target branch.
5. Used git cherry-pick <commit-id>.
6. Resolved any conflicts if required.
7. Verified that the selected commit was applied to the target branch.

```

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ ls
README.md a alphabet amend.txt amend2.txt file2.txt release test webhook-test.txt

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git checkout release
Switched to branch 'release'
Your branch is up to date with 'origin/release'.

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git pull
Already up to date.

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git cherry-pick fb18651a
[release 264db78] adding amend files
Date: Fri Aug 21 15:09:24 2026 +0530
2 files changed, 2 insertions(+)
create mode 100644 amend.txt
create mode 100644 amend2.txt

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git log --oneline
264db78 (HEAD -> release) adding amend files
9331056 (origin/release) resolved the merge conflict of alphabet file

```

```

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git log
commit 264db781f43041ab7d00baba1ee5f991a5c1806c (HEAD -> release, origin/release)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 15:09:24 2026 +0530

    adding amend files

commit 93310563a1c4ed85492be03836dbb61ced4073af
Merge: ad8c6a3 693a67f
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 13:35:22 2026 +0530

    resolved the merge conflict of alphabet file

commit ad8c6a38a0a71334228cebea62ae42bb16bdcf21
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 13:26:37 2026 +0530

    adding alphabet file

commit 693a67ff8dc2ff7df0677f523c5790a8569aad2b (origin/hotfix, hotfix)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 13:20:20 2026 +0530

    modified alphabet file

commit c9595ef61924edae01aa0595b2c14c6b549b1773
Merge: da5e7b4 9090108
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Thu Aug 20 17:46:10 2026 +0530

    Merge branch 'hotfix' of github.com:vamshikumar-a/techie-02 into hotfix

```

release

All users

All time

Commits on Aug 21, 2026

adding amend files

vamshikumar-a committed 2 minutes ago

264db78

<>

Conclusion:

Successfully applied a specific commit from one branch to another using cherry-pick.

6. Interactive Rebase

- Reorder and squash multiple commits into a single clean commit.

Objective:

Reorder and squash multiple commits into a clean commit history.

Purpose:

To understand how interactive rebase can be used to organize and clean up commits.

Implementation:

1. Created multiple commits on a feature branch.
2. Checked the commit history using `git log --oneline`.
3. Started interactive rebase using `git rebase -i HEAD~4`.
4. Reordered the commits.
5. Squashed multiple commits into one.
6. Updated the final commit message.
7. Verified the cleaned history using `git log --oneline --graph`.

```
pick 63f33bb adding release file
squash ad8c6a3 adding alphabet file
squash e690726 added alphabet file
squash e56f493 added test file
```

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git rebase -i HEAD~4
[detached HEAD 7eab49b] modified test file
Date: Fri Aug 21 14:39:51 2026 +0530
3 files changed, 3 insertions(+)
create mode 100644 amend.txt
create mode 100644 amend2.txt
Successfully rebased and updated refs/heads/feature.
```

```
pick f61b107 modified test file
squash 92201ef Revert "added demo file"
squash fb18651 adding amend files

# Rebase 9331056..fb18651 onto 9331056 (4 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                        commit's log message, unless -C is used, in which case
#                        keep only this commit's message; -c is same as -C but
#                        opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
# .      create a merge commit using the original merge commit's
#        message (or the oneline, if no original merge commit was
#        specified); use -C <commit> to reword the commit message
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#
~
```

```
MINGW64:/c/Users/91738/Desktop/techie-02
# This is a combination of 4 commits.
# This is the 1st commit message:

modified test file

# This is the commit message #2:

added demo file

# This is the commit message #3:

Revert "added demo file"

This reverts commit 5343321c929d4b8dfb73f584bddc0bd529fb4a6b.

# This is the commit message #4:

adding amend files

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Fri Aug 21 14:39:51 2026 +0530
#
# interactive rebase in progress; onto 9331056
# Last commands done (4 commands done):
#   squash 92201ef Revert "added demo file"
#   squash fb18691 adding amend files
# No commands remaining.
# You are currently rebasing branch 'feature' on '9331056'.
#
# Changes to be committed:
#   new file:   amend.txt
#   new file:   amend2.txt
#   modified:   test
#
~
~

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (feature)
$ git log
commit 7eab49b32ae916ae794428114545e086290227a5 (HEAD -> feature)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date:   Fri Aug 21 14:39:51 2026 +0530

    modified test file

    added demo file

    Revert "added demo file"

    This reverts commit 5343321c929d4b8dfb73f584bddc0bd529fb4a6b.

    adding amend files

commit 93310563a1c4ed85492be03836dbb61ced4073af
Merge: ad8c6a3 693a67f
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date:   Fri Aug 21 13:35:22 2026 +0530
```

Conclusion:

Successfully reorganized and squashed multiple commits using interactive rebase.

7. Tagging & Release

- Create a version tag (v1.0), push it to GitHub, then delete and restore it.

Objective:

Create a version tag, push it to GitHub, delete it, and restore it.

Purpose:

To understand how Git tags are used to mark application versions and releases.

Implementation:

1. Created a version tag using git tag v1.0.
2. Verified the tag using git tag.
3. Pushed the tag to GitHub.
4. Deleted the local tag.
5. Deleted the remote tag.

6. Recreated the v1.0 tag using the required commit ID.
7. Pushed the restored tag to GitHub.
8. Verified the restored tag.

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git tag v1.0

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git log
commit 264db781f43041ab7d00baba1ee5f991a5c1806c (HEAD -> release, tag: v1.0, origin/release)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 15:09:24 2026 +0530

    adding amend files

commit 93310563a1c4ed85492be03836dbb61ced4073af
Merge: ad8c6a3 693a67f
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 13:35:22 2026 +0530

    resolved the merge conflict of alphabet file
```

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git push origin v1.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:vamshikumar-a/techie-02.git
 * [new tag]          v1.0 -> v1.0

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git tag -d v1.0
Deleted tag 'v1.0' (was 264db78)

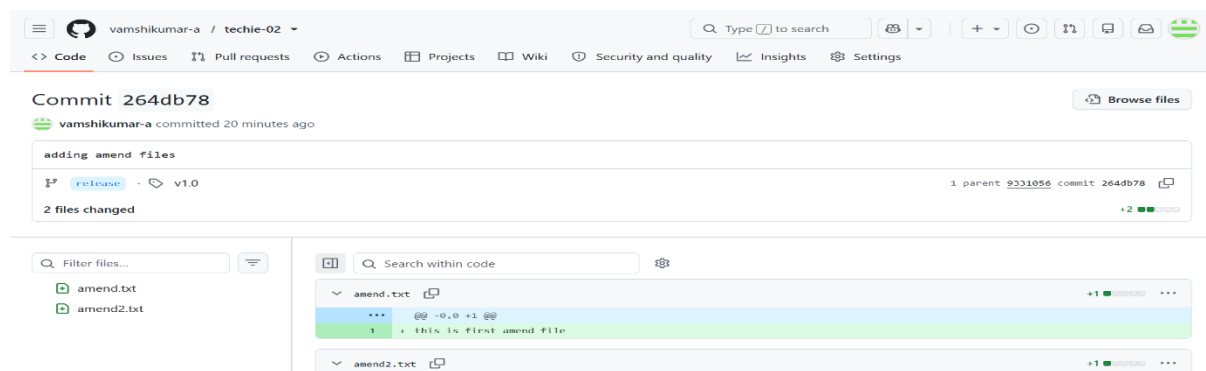
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git push origin --delete v1.0
To github.com:vamshikumar-a/techie-02.git
 - [deleted]          v1.0

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git log
commit 264db781f43041ab7d00baba1ee5f991a5c1806c (HEAD -> release, origin/release)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 15:09:24 2026 +0530

    adding amend files

commit 93310563a1c4ed85492be03836dbb61ced4073af
Merge: ad8c6a3 693a67f
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 13:35:22 2026 +0530

    resolved the merge conflict of alphabet file
```



The screenshot shows the GitHub web interface for the repository 'vamshikumar-a / techie-02'. The commit 264db78 is displayed, with the message 'adding amend files'. The commit was made by 'vamshikumar-a' 20 minutes ago. The diff shows two files changed: 'amend.txt' and 'amend2.txt'. The 'amend.txt' diff shows a new line added: '1 + this is first amend file'.

Commit 264db78

 vamshikumar-a committed 23 minutes ago

adding amend files

 [release](#)

1 parent [93316](#)

2 files changed

Filter files...

 amend.txt

Search within code

 amend.txt 

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git tag v1.0 264db78

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git push origin v1.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:vamshikumar-a/techie-02.git
 * [new tag]          v1.0 -> v1.0

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (release)
$ git log
commit 264db781f43041ab7d00baba1ee5f991a5c1806c (HEAD -> release, tag: v1.0, origin/release)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 15:09:24 2026 +0530

    adding amend files
```

Conclusion:

Successfully created, pushed, deleted, and restored a Git version tag.

8. Clone with Sparse Checkout

- Clone only a subdirectory of a repo using sparse checkout.

Objective:

Clone a repository and check out only a specific subdirectory.

Purpose:

To work with only the required portion of a repository instead of checking out the complete repository.

Implementation

1. Clone the repository without checking out the files:
`git clone --no-checkout git@github.com:vamshikumar-a/techie-02.git`
2. Navigate to the cloned repository:
`cd techie-02`
3. Initialize sparse checkout in non-cone mode:
`git sparse-checkout init --no-cone`
4. Configure Git to check out only the sample/ directory:
`git sparse-checkout set --no-cone 'sample/'`
5. Update the working tree using:

git read-tree -mu HEAD

6. Verify the checked-out files: ls
7. Confirm that only the sample/ directory is available in the working directory:
8. Verify the sparse-checkout configuration:
9. Verify that the sample directory exists in the repository:
10. The final working directory contains only the required sample/ directory instead of the complete repository.

```
91738@IDEAPAD MINGW64 ~/Desktop/techieclone
$ ls

91738@IDEAPAD MINGW64 ~/Desktop/techieclone
$ git clone --no-checkout git@github.com:vamshikumar-a/techie-02.git
Cloning into 'techie-02'...
remote: Enumerating objects: 55, done.
remote: Counting objects: 100% (55/55), done.
remote: Compressing objects: 100% (32/32), done.
remote: Total 55 (delta 23), reused 37 (delta 9), pack-reused 0 (from 0)
Receiving objects: 100% (55/55), 7.09 KiB | 330.00 KiB/s, done.
Resolving deltas: 100% (23/23), done.

91738@IDEAPAD MINGW64 ~/Desktop/techieclone
$ ls
techie-02/
```

```
91738@IDEAPAD MINGW64 ~/Desktop/techieclone/techie-02 (main|SPARSE)
$ git sparse-checkout set --no-cone 'sample/'

91738@IDEAPAD MINGW64 ~/Desktop/techieclone/techie-02 (main|SPARSE)
$ git read-tree -mu HEAD

91738@IDEAPAD MINGW64 ~/Desktop/techieclone/techie-02 (main|SPARSE)
$ ls -la
total 4
drwxr-xr-x 1 91738 197609 0 Aug 21 16:29 ./
drwxr-xr-x 1 91738 197609 0 Aug 21 16:10 ../
drwxr-xr-x 1 91738 197609 0 Aug 21 16:30 .git/
drwxr-xr-x 1 91738 197609 0 Aug 21 16:29 sample/

91738@IDEAPAD MINGW64 ~/Desktop/techieclone/techie-02 (main|SPARSE)
$ ls
sample/
```

Conclusion:

Successfully used sparse checkout to work with a specific repository subdirectory.

9. Reset vs Revert Challenge

- Demonstrate the difference between git reset --hard and git revert in a repo.

Objective:

Demonstrate the difference between git reset --hard and git revert.

Purpose:

To understand two different approaches for undoing Git changes.

Implementation:

1. Created and committed a test change.
2. Used git reset --hard HEAD~1.
3. Verified that the branch moved back to the previous commit.
4. Created another test commit.
5. Used git revert <commit-id>.
6. Checked the commit history.
7. Compared the results of reset and revert.

```

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ ls
README.md a alphabet b demo/ directory/ sample/ test

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git pull
Already up to date.

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ nano test

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test

no changes added to commit (use "git add" and/or "git commit -a")

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git add .
warning: in the working copy of 'test', LF will be replaced by CRLF the next time Git touches it

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git commit -m "modifying test file"
[main a5bf578] modifying test file
1 file changed, 1 insertion(+)

```

```

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git reset --hard HEAD~1
HEAD is now at aeb4703 adding the folders

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ ls
README.md a alphabet b demo/ directory/ sample/ test

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ cat test

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git log
commit aeb470348cdccd492071cd5c46d281a20ffd5cd9 (HEAD -> main, origin/main, origin/HEAD)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date:   Fri Aug 21 16:08:44 2026 +0530

    adding the folders

```

```

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git log
commit a5bf5782c9a128a9f7c9ce0f37042cca4d83e310 (HEAD -> main)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 17:17:04 2026 +0530

    modifying test file

commit aeb470348cdccd492071cd5c46d281a20ffd5cd9 (origin/main, origin/HEAD)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 16:08:44 2026 +0530

    adding the folders

commit 7f7be19c48e71c41dd29dcc254b5ca2225d17fac
Merge: e690726 248d37e
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Thu Aug 20 15:44:44 2026 +0530

    Merge pull request #1 from vamshikumar-a/hotfix

    added test file

commit 248d37eccc5bd35019a1794913144ab14fbf6c2c
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Thu Aug 20 15:42:18 2026 +0530

    added test file

```

```

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git revert HEAD
[main dcc5388] Revert "modified test file"
1 file changed, 1 deletion(-)

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git log
commit dcc53883fd0164c899e396593762d01fe913b78f (HEAD -> main)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 17:22:39 2026 +0530

    Revert "modified test file"

    This reverts commit b82eb2b87812216f3d369d27a0feec005995130a.

commit b82eb2b87812216f3d369d27a0feec005995130a
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 17:21:57 2026 +0530

    modified test file

commit aeb470348cdccd492071cd5c46d281a20ffd5cd9 (origin/main, origin/HEAD)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Fri Aug 21 16:08:44 2026 +0530

    adding the folders

```

Conclusion:

Successfully demonstrated that reset --hard moves the branch backward, while revert creates a new commit to undo the previous change.

10. Detached HEAD Challenge

- Checkout a specific commit (detached HEAD state) and create a new branch from it.

Objective:

Checkout a specific commit and create a new branch from the detached HEAD state.

Purpose:

To understand the detached HEAD state and recover or continue development from a specific commit.

Implementation:

1. Checked the commit history using git log.
2. Checked out a specific commit using git checkout <commit-id>.

3. Verified the detached HEAD state using git status.
4. Created a new branch from the current commit.
5. Switched to the newly created branch.
6. Verified the branch and commit history.

```

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ ls
README.md a alphabet b demo/ directory/ sample/ test
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ nano test
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ cat test
this is test file
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test

no changes added to commit (use "git add" and/or "git commit -a")
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git add .
warning: in the working copy of 'test', LF will be replaced by CRLF the next time Git touches it
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git commit -m "modified test file"
[main b82eb2b] modified test file
1 file changed, 1 insertion(+)
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git log
commit b82eb2b87812216f3d369d27a0feec005995130a (HEAD -> main)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date:   Fri Aug 21 17:21:57 2026 +0530

    modified test file

commit aeb470348cdcc492071cd5c46d281a20ffd5cd9 (origin/main, origin/HEAD)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date:   Fri Aug 21 16:08:44 2026 +0530

    adding the folders

```

```

MINGW64/c/Users/91738/Desktop/techie-02
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git log --oneline
dc5388 (HEAD -> main, origin/main, origin/HEAD) Revert "modified test file"
b82eb2b modified test file
aeb4703 adding the folders
77fb19 Merge pull request #1 from vamshikumar-a/hotfix
248d37e added test file
e690726 added alphabet file
2dfb5d1 added a,b files
9cd8c92 Initial commit
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git checkout aeb4703
Note: switching to 'aeb4703'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at aeb4703 adding the folders
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 ((aeb4703...))
$ git checkout -b recovery-branch
Switched to a new branch 'recovery-branch'
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovery-branch)
$ git branch
  feature
  hotfix
* main
+ recovery-branch
  release
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovery-branch)
$ git log
commit aeb470348cdcc492071cd5c46d281a20ffd5cd9 (HEAD -> recovery-branch)
Author: vamshikumar-a <vamshikumara1004@gmail.com>

```

```

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovery-branch)
$ git log
commit aeb470348cdccd492071cd5c46d281a20ffd5cd9 (HEAD -> recovery-branch)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date:   Fri Aug 21 16:08:44 2026 +0530

    adding the folders

commit 7f7be19c48e71c41dd29dcc254b5ca2225d17fac
Merge: e690726 248d37e
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date:   Thu Aug 20 15:44:44 2026 +0530

    Merge pull request #1 from vamshikumar-a/hotfix

    added test file

commit 248d37eccc5bd35019a1794913144ab14fbf6c2c
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date:   Thu Aug 20 15:42:18 2026 +0530

    added test file

commit e690726515044534681edc16d5e7d6baa973747e
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date:   Thu Aug 20 15:39:03 2026 +0530

    added alphabet file

```

Commits

recovery-branch
All users
All time

Commits on Aug 21, 2026

adding the folders
vamshikumar-a committed 1 hour ago
aeb4703

Commits on Aug 20, 2026

Merge pull request #1 from vamshikumar-a/hotfix
Verified
7f7be19

added test file
vamshikumar-a committed yesterday
248d37e

Conclusion:

Successfully created a new branch from a detached HEAD commit.

11. Git Hooks Challenge

- Configure a pre-commit hook to reject commits without a message format (e.g., must start with JIRA-XXX).

Objective:

Configure a Git hook to reject commits that do not follow a required message format.

Purpose:

To automatically enforce a standard commit-message format.

Implementation:

1. Created a commit-msg hook inside .git/hooks.
2. Added validation to check that the commit message starts with JIRA-.
3. Made the hook executable using chmod +x.
4. Attempted a commit with an invalid message.

5. Verified that the commit was rejected.
6. Created a valid commit such as JIRA-123 Fix login issue.
7. Verified that the valid commit was accepted.

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovery-branch)
$ nano .git/hooks/commit-msg

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovery-branch)
$ chmod +x .git/hooks/commit-msg

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovery-branch)
$ git status
On branch recovery-branch
nothing to commit, working tree clean
```

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovery-branch)
$ echo "Testing commit hook" > hook-test.txt

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovery-branch)
$ git status
On branch recovery-branch
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        hook-test.txt

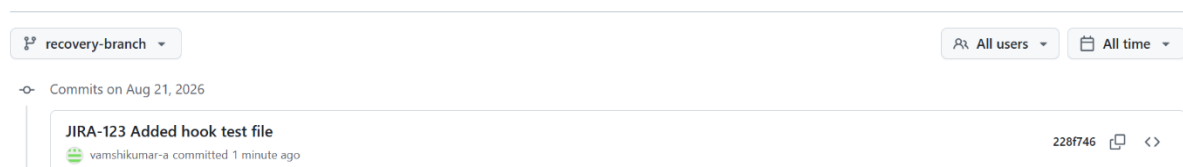
nothing added to commit but untracked files present (use "git add" to track)

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovery-branch)
$ git add hook-test.txt
warning: in the working copy of 'hook-test.txt', LF will be replaced by CRLF the next time Git touches it

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovery-branch)
$ git status
On branch recovery-branch
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   hook-test.txt

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovery-branch)
$ git commit -m "Added hook test file"
ERROR: Commit message must start with JIRA-XXX

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovery-branch)
$ git commit -m "JIRA-123 Added hook test file"
[recovery-branch 228f746] JIRA-123 Added hook test file
1 file changed, 1 insertion(+)
create mode 100644 hook-test.txt
```



Conclusion:

Successfully configured a Git hook to enforce the required JIRA commit-message format.

12. Squash Merge vs Rebase Merge

- Show the difference between squash merge and rebase merge with evidence.

Objective:

Demonstrate the difference between squash merge and rebase merge with Git history evidence.

Purpose:

To understand how different merge strategies affect the Git commit history.

Implementation:

1. Created a feature branch.
2. Created multiple commits on the feature branch.
3. Pushed the branch to GitHub.
4. Created a Pull Request.
5. Used **Squash and merge** and checked the resulting history.
6. Created another feature branch with multiple commits.
7. Used **Rebase and merge**.
8. Checked the history using `git log --oneline --graph --all`.
9. Compared both merge results.

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git branch rebase-merge

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git branch --list
feature
hotfix
* main
rebase-merge
recovery-branch
release
squase-merge

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git checkout re
rebase-merge      recovery-branch  release

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git checkout re
rebase-merge      recovery-branch  release

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git checkout rebase-merge
Switched to branch 'rebase-merge'

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (rebase-merge)
$ ls
README.md a alphabet b demo/ directory/ sample/ test

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (rebase-merge)
$ touch rebase1

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (rebase-merge)
$ git add .

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (rebase-merge)
$ git commit -m "JIRA-123 added rebase1 file"
[rebase-merge 0482a96] JIRA-123 added rebase1 file
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 rebase1
```

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (squase-merge)
$ touch squash2

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (squase-merge)
$ git add .

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (squase-merge)
$ git commit -m "JIRA-123 added squash2 file"
[squase-merge 0482a96] JIRA-123 added squash2 file
1 file changed, 0 insertions(+), 0 deletions(-)
Create mode 100644 squash2

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (squase-merge)
$ git push origin squase-merge
Enumerating objects: 3, done.
Counting objects: 100%, 0/3, done.
Delta compression using up to 8 threads
Compressing objects: 100%, 0/23, done.
Writing objects: 100%, 0/23, 11.00 KiB/s, done.
Total 0 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100%, 0/2, completed with 1 local object.
to github.com:vamshikumar-a/techie-02.git
ef83a7..c07d09 squase-merge -> squase-merge

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (squase-merge)
$ touch squash3

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (squase-merge)
$ git add .

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (squase-merge)
$ git commit -m "JIRA-123 added squash3 file"
[squase-merge 0482a96] JIRA-123 added squash3 file
1 file changed, 0 insertions(+), 0 deletions(-)
Create mode 100644 squash3

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (squase-merge)
$ git push origin squase-merge
Enumerating objects: 3, done.
Counting objects: 100%, 0/3, done.
Delta compression using up to 8 threads
Compressing objects: 100%, 0/23, done.
Writing objects: 100%, 0/23, 11.00 KiB/s, done.
Total 0 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100%, 0/2, completed with 1 local object.
to github.com:vamshikumar-a/techie-02.git
c47d00..67b0e7 squase-merge -> squase-merge
```

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (squase-merge)
$ git log
commit 6fbbcef5463e907d13e044874672285da86884bd (HEAD -> squase-merge, origin/squase-merge)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Sun Aug 23 08:47:10 2026 +0530

    JIRA-123 added squash3 file

commit ce70d09896c737df314cc761228ce33a103d356b
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Sun Aug 23 08:46:33 2026 +0530

    JIRA-123 added squash2 file

commit ef68ad71bd9bb640a55208c6f7d408705da3d0a0
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Sun Aug 23 08:44:57 2026 +0530

    JIRA-123 added squash1 file

commit dcc53883fd0164e889e20c582762d01fa913b78f (origin/main, origin/HEAD, main)
```

The screenshot shows the GitHub Pull Request #2 interface. The title is "Squase merge #2" and the description is "vamshikumar-a wants to merge 3 commits into main from squase-merge". The commit message is "Squase merge (#2)" and the extended description lists three commits: "JIRA-123 added squash1 file", "JIRA-123 added squash2 file", and "JIRA-123 added squash3 file". The interface includes a "Confirm squash and merge" button, a "Cancel" button, and a "Still in progress? Convert to draft" link. The right sidebar shows the milestone, development status, notifications, and participants.

The screenshot shows the GitHub Pull Request #2 interface. The title is "Squase merge #2" and the description is "vamshikumar-a wants to merge 3 commits into main from squase-merge". The commit message is "Squase merge (#2)" and the extended description lists three commits: "JIRA-123 added squash1 file", "JIRA-123 added squash2 file", and "JIRA-123 added squash3 file". The interface includes a "Squase merge" button, a "Cancel" button, and a "Still in progress? Convert to draft" link. The right sidebar shows the labels, projects, milestones, development status, notifications, and participants.

Commits

The screenshot shows the GitHub Commits page. The main branch is "main". The commit history is filtered by "All users" and "All time". The commit history shows three commits: "Squase merge (#2)" by vamshikumar-a, "Revert 'modified test file'" by vamshikumar-a, and "modified test file" by vamshikumar-a.

```

91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git log
commit a5120a4f56ee978bb45fae73ea1115c7c5b84653 (HEAD -> main, origin/main, origin/HEAD)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Sun Aug 23 09:23:17 2026 +0530

Squase merge (#2)

* JIRA-123 added squash1 file

* JIRA-123 added squash2 file

* JIRA-123 added squash3 file

commit dcc53883fd0164c899e396593762d01fe913b78f (master)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Sun Aug 23 17:22:30 2026 +0530

```

```

MINGW64/C:/Users/91738/Desktop/techie-02
$ git push origin rebase-merge
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 276 bytes | 276.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
remote: Create a pull request for 'rebase-merge' on GitHub by visiting:
remote:   https://github.com/vamshikumar-a/techie-02/pull/new/rebase-merge
To github.com:vamshikumar-a/techie-02.git
 * [new branch]      rebase-merge -> rebase-merge
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (rebase-merge)
$ touch rebase2
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (rebase-merge)
$ git add .
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (rebase-merge)
$ git commit -m "JIRA-123 added rebase2 file"
[rebase-merge f0471a3] JIRA-123 added rebase2 file
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 rebase2
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (rebase-merge)
$ touch rebase3
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (rebase-merge)
$ git push origin rebase-merge
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 256 bytes | 256.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:vamshikumar-a/techie-02.git
 * 0482a96:f0471a3 rebase-merge -> rebase-merge
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (rebase-merge)
$ git add .
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (rebase-merge)
$ git commit -m "JIRA-123 added rebase3 file"

```

```

git log --oneline --graph --all
35ea495 (HEAD -> main, origin/main, origin/HEAD) JIRA-123 added rebase3 file
ab63713 JIRA-123 added rebase2 file
e7e6b72 JIRA-123 added rebase1 file
70a4b7b JIRA-123 added abd file
a5120a4 Squase merge (#2)
* fdd7849 (origin/rebase-merge, rebase-merge) JIRA-123 added rebase3 file
* f0471a3 JIRA-123 added rebase2 file
* 0482a96 JIRA-123 added rebase1 file
/
* 6fbbcef (origin/squase-merge, squase-merge) JIRA-123 added squash3 file

```

devops-cloud-22 (Channel) | Techie Horizon - Karan B... | Inbox - vamshikumara1004 | Rebase merge by vamshikumar-a | Ask Gemini | Finish update

github.com/vamshikumar-a/techie-02/pull/3

Rebase merge #3
vamshikumar-a wants to merge 3 commits into `main` from `rebase-merge`

vamshikumar-a added 3 commits 17 minutes ago

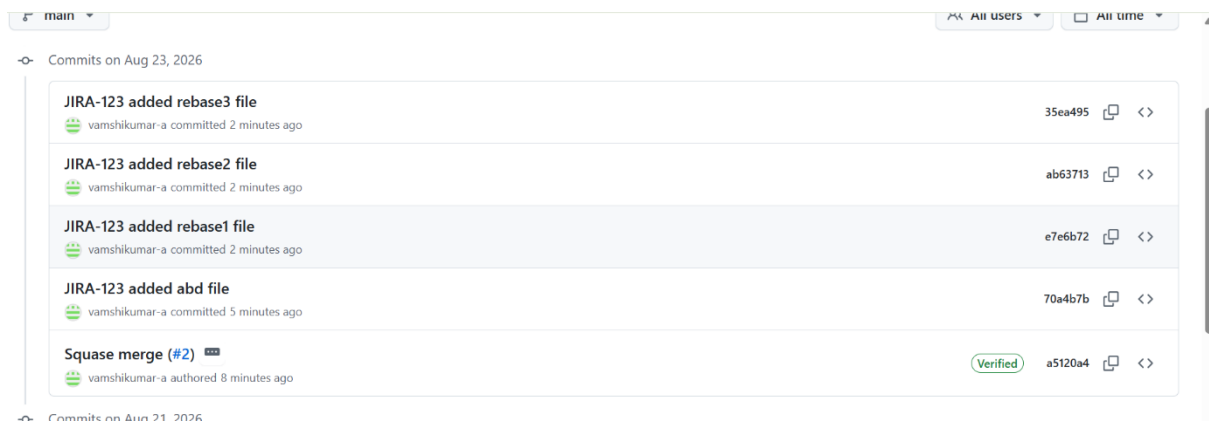
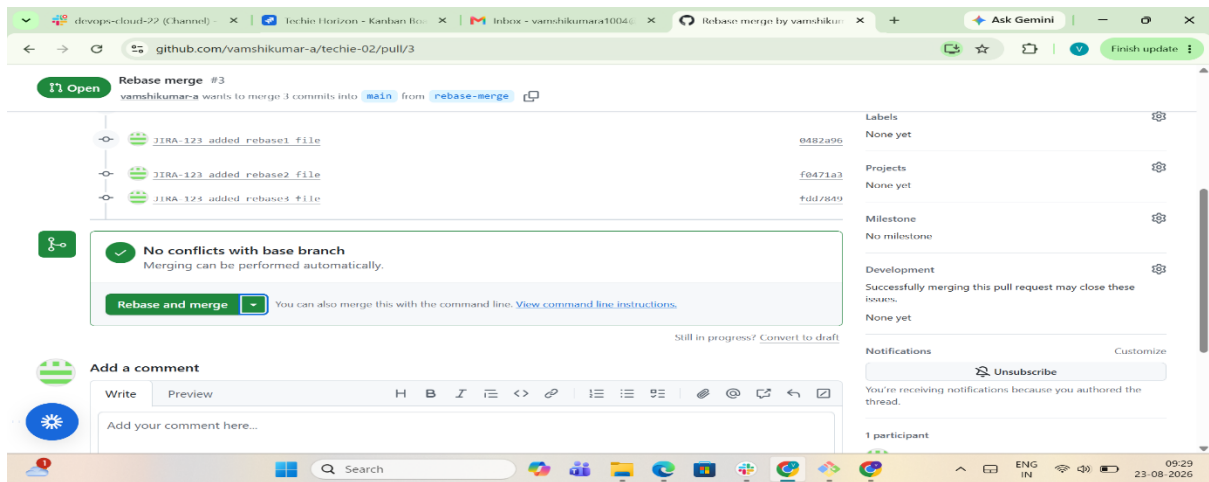
- JIRA-123 added rebase1 file (0482a96)
- JIRA-123 added rebase2 file (f0471a3)
- JIRA-123 added rebase3 file (fdd7849)

This will rebase your changes and merge them into main.

Still in progress? Convert to draft

Add a comment
Write | Preview

Assignees: No one—assign yourself
Labels: None yet
Projects: None yet
Milestone: No milestone
Development: Successfully merging this pull request may close these issues. None yet
Notifications: Unsubscribe | Customize



```

$ git log
commit 35ea4954e9227bf7acd11d2c88cc44f0b8e55fc9 (HEAD -> main, origin/main, origin/HEAD)
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Sun Aug 23 09:13:04 2026 +0530

    JIRA-123 added rebase3 file

commit ab6371350caa2bf8ebcea4eca2c5f807a7d6b36a
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Sun Aug 23 09:12:26 2026 +0530

    JIRA-123 added rebase2 file

commit e7e6b72a48790d348ab63e7f0edd756142324817
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Sun Aug 23 09:11:43 2026 +0530

    JIRA-123 added rebase1 file

commit 70a4b7b3d9775d66d44f3c7194b0d4eb7268e84e
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Sun Aug 23 09:26:55 2026 +0530

    JIRA-123 added abd file

commit a5120a4f56ee978bb45fae73ea1115c7c5b84653
Author: vamshikumar-a <vamshikumara1004@gmail.com>
Date: Sun Aug 23 09:23:17 2026 +0530

    Squase merge (#2)

    * JIRA-123 added squash1 file
    * JIRA-123 added squash2 file
    * JIRA-123 added squash3 file
  
```

Conclusion:

Successfully demonstrated that **squash merge** combines multiple commits into one, while **rebase merge** keeps the individual commits in a linear history.

13. Fork & Pull Request Workflow

- Fork a repo, make a change, and submit a pull request to the original repo.

Objective:

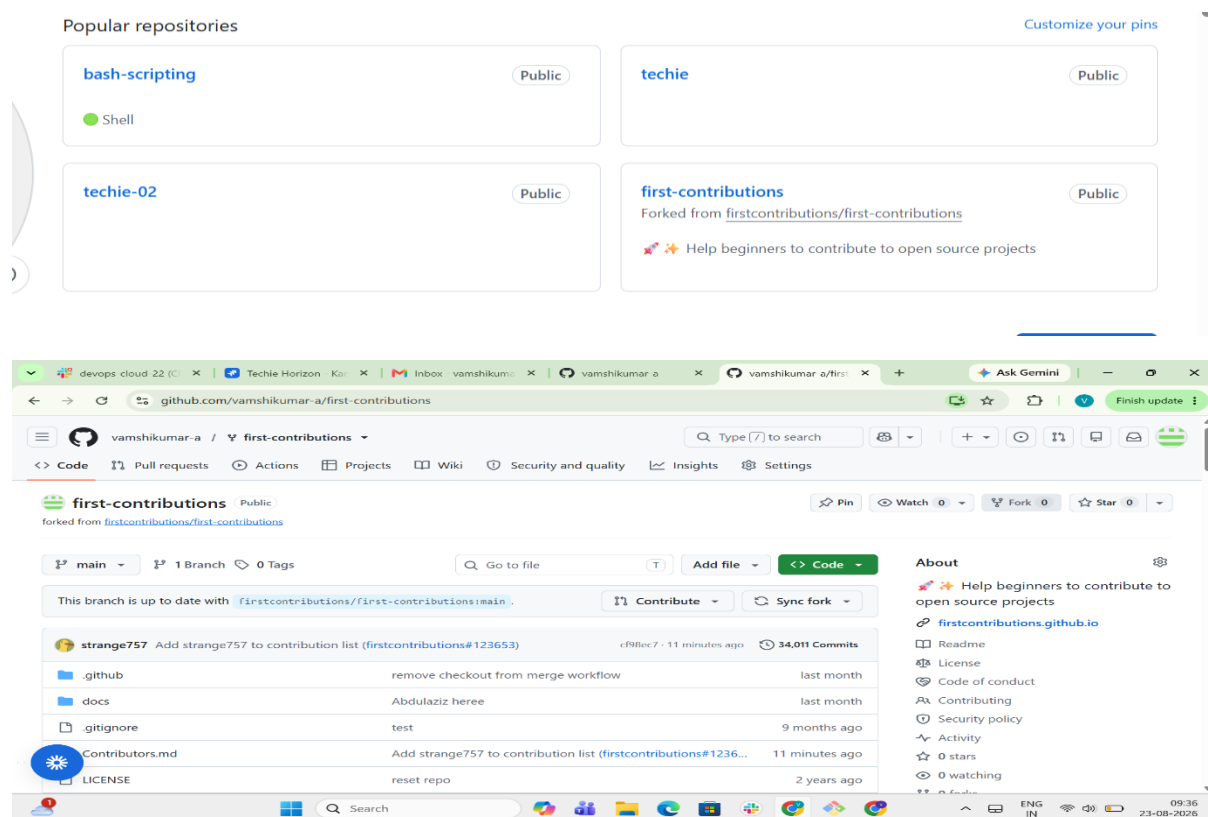
Fork an open-source repository, make a change, and submit a Pull Request to the original repository.

Purpose:

To understand the GitHub contribution workflow used for open-source projects.

Implementation:

1. Selected an open-source GitHub repository.
2. Forked the repository to the GitHub account.
3. Cloned the forked repository locally.
4. Created a new feature branch.
5. Made the required changes.
6. Added and committed the changes.
7. Pushed the feature branch to the forked repository.
8. Created a Pull Request from the fork to the original repository.
9. Reviewed the changes and submitted the Pull Request.



```

91738@IDEAPAD MINGW64 ~/Desktop
$ git clone git@github.com:vamshikumar-a/first-contributions.git
Cloning into 'first-contributions'...
remote: Enumerating objects: 93610, done.
remote: Counting objects: 100% (225/225), done.
remote: Compressing objects: 100% (27/27), done.
remote: Total 93610 (delta 222), reused 198 (delta 198), pack-reused 93385 (from 4)
Receiving objects: 100% (93610/93610), 45.92 MiB | 6.12 MiB/s, done.
Resolving deltas: 100% (60957/60957), done.

91738@IDEAPAD MINGW64 ~/Desktop
$ ls
'S class half yearly question paper.docx'  'MongoDB Compass.lnk'*          'Science_Question_Paper (1).pdf'  techie-02/
'A VAMSHI KUMAR (2).docx'                  'Movavi Video Editor 25.lnk'*    'Visual Studio Code.lnk'*        techieclone/
'A VAMSHI-MERN-STACK.pdf'                  'New Microsoft Excel Worksheet (2).xlsx'  'Zoom Workplace.lnk'*            vamshi_projects/
'ASV PDF.pdf'                             'New Microsoft Excel Worksheet.xlsx'      desktop.ini                       wordpress/
'ASV-Apartment Unpriced 80Q.xlsx'          'PLY WOOD SHUTTERING QUOTATION.pdf'      first-contributions/              ~$New Microsoft Excel Worksheet (2)
'House-plan/'                             'Science_Question_Paper (1) (2).pdf'      techie/
'Marriage items.xlsx'                    'Science_Question_Paper (1).docx'

```

```

91738@IDEAPAD MINGW64 ~/Desktop
$ cd first-contributions/

91738@IDEAPAD MINGW64 ~/Desktop/first-contributions (main)
$ ls
Contributors.md  LICENSE  README.md  docs/

91738@IDEAPAD MINGW64 ~/Desktop/first-contributions (main)
$ git branch feature

91738@IDEAPAD MINGW64 ~/Desktop/first-contributions (main)
$ git branch --list
* main
  feature

91738@IDEAPAD MINGW64 ~/Desktop/first-contributions (main)
$ git checkout feature
Switched to branch 'feature'

91738@IDEAPAD MINGW64 ~/Desktop/first-contributions (feature)
$ ls
Contributors.md  LICENSE  README.md  docs/

```

```

MINGW64/c:/Users/91738/Desktop/first-contributions
91738@IDEAPAD MINGW64 ~/Desktop/first-contributions (feature)
$ git checkout feature
Switched to branch 'feature'

91738@IDEAPAD MINGW64 ~/Desktop/first-contributions (feature)
$ ls
Contributors.md  LICENSE  README.md  docs/

91738@IDEAPAD MINGW64 ~/Desktop/first-contributions (feature)
$ vi Contributors.md

91738@IDEAPAD MINGW64 ~/Desktop/first-contributions (feature)
$ git status
On branch feature
Changes not staged for commit:
  (use "git add ..." to update what will be committed)
  (use "git restore ..." to discard changes in working directory)
        modified:   Contributors.md

no changes added to commit (use "git add" and/or "git commit -a")

91738@IDEAPAD MINGW64 ~/Desktop/first-contributions (feature)
$ git add .

91738@IDEAPAD MINGW64 ~/Desktop/first-contributions (feature)
$ git commit -m "modified contributors file"
[feature 45d35625] modified contributors file
 1 file changed, 1 insertion(+)

91738@IDEAPAD MINGW64 ~/Desktop/first-contributions (feature)
$ git push origin feature
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), 322 bytes | 322.00 KiB/s, done.
Writing objects: 100% (3/3), 322 bytes | 322.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote: Create a pull request for 'feature' on GitHub by visiting:
remote:   https://github.com/vamshikumar-a/first-contributions/pull/new/feature
remote: To github.com:vamshikumar-a/first-contributions.git
+ [new branch]      feature -> feature

91738@IDEAPAD MINGW64 ~/Desktop/first-contributions (feature)
$ !

```

devops cloud 22 | Techie Horizon | Inbox | vamshikuma... | modified contribu... | Ask Gemini | Finish update

github.com/firstcontributions/first-contributions/pull/123678

firstcontributions / first-contributions

modified contributors file #123678

Open vamshikumar-a wants to merge 1 commit into firstcontributions:main from vamshikumar-a:feature

Conversation | Commits 1 | Checks 0 | Files changed 1

vamshikumar-a commented now

Before submitting this pull request, check the changes to see it's only the changes you made intentionally. If there are changes to other lines you didn't make deliberately, it's possible that your IDE made the changes with a utility like prettier.

Next time, make sure that you only add your changes by using `git add -p` and rather than `git add Contributors.md`

If you're doing something in the checklist below, put an `x` inside `[ ]` so that `- [ ]` becomes `- [x]`

- ☐ I had fun going through this tutorial (/ ^ o ^) / and learned on the way ^ _ ^ ^ ^
- ☐ There are some things I'd like to improve in this tutorial. I have written them below.
- ☐ There were steps where I had errors while following this tutorial. I have written them below.

Reviewers

No reviews

Still in progress? Convert to draft

Assignees

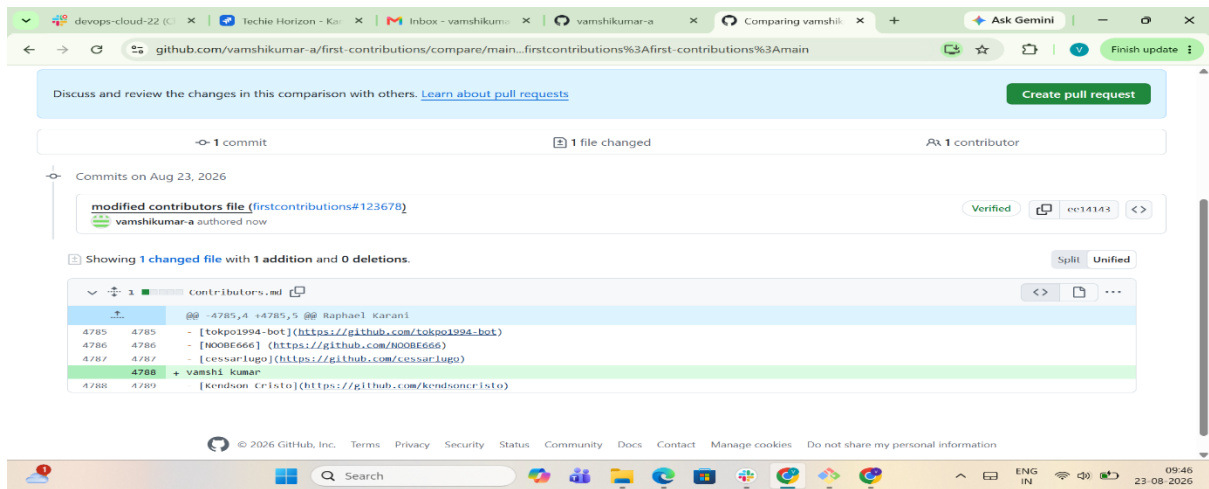
No one assigned

Labels

None yet

Projects

None yet



Conclusion:

Successfully completed the Fork and Pull Request workflow.

14. Recover Lost Commit

- Commit something, reset hard, and then recover it using git reflog.

Objective:

Recover a lost commit after performing git reset --hard using git reflog.

Purpose:

To learn how Git reflog can be used to recover accidentally removed commits.

Implementation:

1. Created a file and committed it.
2. Verified the commit using git log.
3. Used git reset --hard HEAD~1.
4. Verified that the commit was no longer visible in the normal Git history.
5. Used git reflog to find the previous commit.
6. Identified the lost commit ID.
7. Created a new branch from the recovered commit.
8. Verified the recovered file and commit history.

```
MINGW64/c/Users/91738/Desktop/techie-02
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ ls
README.md a adb alphabet b demo/ directory/ rebase1 rebase2 rebase3 sample/ squash1 squash2 squash3 test
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ echo "Important data" > lost.txt
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ cat lost.txt
Important data
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git add lost.txt
warning: in the working copy of 'lost.txt', LF will be replaced by CRLF the next time Git touches it
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git commit -m "Add important data"
ERROR: Commit message must start with JIRA-XXX
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git commit -m "JIRA-123 Add important data"
[main 98510a7] JIRA-123 Add important data
1 file changed, 1 insertion(+)
create mode 100644 lost.txt
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git log --one-line
98510a7 (HEAD -> main) JIRA-123 Add important data
35ea495 (origin/main, origin/HEAD) JIRA-123 added rebase3 file
ab63713 JIRA-123 added rebase2 file
a7e6b72 JIRA-123 added rebase1 file
70a4b7b JIRA-123 added abd file
a51204a Squase merge (#2)
dc53388 (master) Revert "modified test file"
b82eb2b modified test file
aeb4703 adding the folders
7f7be19 Merge pull request #1 from vamshikumar-a/hotfix
248d37e added test file
a69072e added alphabet file
2dfb5d1 added a,b files
9cd8c92 Initial commit
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git reset --hard HEAD~1
HEAD is now at 35ea495 JIRA-123 added rebase3 file
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
```

```
MINGW64/c/Users/91738/Desktop/techie-02
$ git log --one-line
98510a7 (HEAD -> main, origin/main, origin/HEAD) JIRA-123 added rebase3 file
ab63713 JIRA-123 added rebase2 file
a7e6b72 JIRA-123 added rebase1 file
70a4b7b JIRA-123 added abd file
a51204a Squase merge (#2)
dc53388 (master) Revert "modified test file"
b82eb2b modified test file
aeb4703 adding the folders
7f7be19 Merge pull request #1 from vamshikumar-a/hotfix
248d37e added test file
a69072e added alphabet file
2dfb5d1 added a,b files
9cd8c92 Initial commit
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ ls
README.md a adb alphabet b demo/ directory/ rebase1 rebase2 rebase3 sample/ squash1 squash2 squash3 test
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git reflog
98510a7 HEAD -> main, origin/main, origin/HEAD HEAD@{0}: reset: moving to HEAD~1
98510a7 HEAD@{1}: commit: JIRA-123 Add important data
35ea495 HEAD@{2}: pull origin main: Fast-forward
70a4b7b HEAD@{3}: checkout: moving from main to main
70a4b7b HEAD@{4}: checkout: moving from rebase-merge to main
fd7849 (origin/rebase-merge, rebase-merge) HEAD@{5}: checkout: moving from main to rebase-merge
70a4b7b HEAD@{6}: commit: JIRA-123 added abd file
a51204a HEAD@{7}: pull origin main: Fast-forward
dc53388 (master) HEAD@{8}: checkout: moving from squash-merge to main
b82eb2b (origin/squase-merge, squash-merge) HEAD@{9}: checkout: moving from main to squash-merge
dc53388 (master) HEAD@{10}: checkout: moving from rebase-merge to main
fd7849 (origin/rebase-merge, rebase-merge) HEAD@{11}: commit: JIRA-123 added rebase3 file
0482a90 HEAD@{12}: commit: JIRA-123 added rebase2 file
dc53388 HEAD@{13}: checkout: moving from main to rebase-merge
dc53388 (master) HEAD@{14}: checkout: moving from squash-merge to main
b82eb2b (origin/squase-merge, squash-merge) HEAD@{15}: commit: JIRA-123 added squash3 file
c970d09 HEAD@{16}: commit: JIRA-123 added squash2 file
f66a047 HEAD@{17}: checkout: moving from recovery-branch to main
dc53388 (master) HEAD@{18}: checkout: moving from main to squash-merge
228f74e (origin/recovery-branch, recovery-branch) HEAD@{19}: commit: JIRA-123 Added hook test file
aeb4703 HEAD@{20}: checkout: moving from aeb7054e4cdcd482071cd5c46dd261a20ffdcde to recovery-branch
aeb4703 HEAD@{21}: checkout: moving from main to aeb4703
dc53388 (master) HEAD@{22}: revert: revert "modified test file"
b82eb2b HEAD@{23}: commit: modified test file
aeb4703 HEAD@{24}: reset: moving to HEAD~1
```

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (main)
$ git checkout -b recovered-branch 98510a7
Switched to a new branch 'recovered-branch'
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovered-branch)
$ ls
README.md a adb alphabet b demo/ directory/ lost.txt rebase1 rebase2 rebase3 sample/ squash1 squash2 squash3 test
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovered-branch)
$ |
```

```
91738@IDEAPAD MINGW64 ~/Desktop/techie-02 (recovered-branch)
$ git log --one-line
98510a7 (HEAD -> recovered-branch) JIRA-123 Add important data
35ea495 (origin/main, origin/HEAD, main) JIRA-123 added rebase3 file
ab63713 JIRA-123 added rebase2 file
e7e6b72 JIRA-123 added rebase1 file
70a4b7b JIRA-123 added abd file
```

Conclusion:

Successfully recovered a lost commit using git reflog.